

¿Qué es una API REST?

Teoría, analogías y ejemplos reales

Para estudiantes de Ingeniería

{ API }

REST

La gran analogía: el restaurante 🍴



Vos (Cliente)

el comensal

Hace un pedido usando
el menú (interfaz)
sin saber cómo se
cocinan los platos.



El Mozo

la API

Toma el pedido,
lo lleva a cocina y
te trae la respuesta.
No cocina ni decide.



La Cocina

el servidor / BD

Procesa la solicitud,
prepara el dato
y lo devuelve.
El cliente no la ve.



La API es el mozo: define QUÉ podés pedir, CÓMO pedirlo y te devuelve el resultado. El cliente nunca toca la cocina.

¿Qué significa REST?

RE

REpresentational

Los datos se representan en un formato estándar (JSON, XML). El servidor no manda pantallas, manda datos.

S

State

Cada respuesta transporta el ESTADO del recurso en ese momento (ej: el usuario tal como está en la BD ahora).

T

Transfer

Esa representación se transfiere a través de la red usando el protocolo HTTP.

+ HTTP como protocolo de comunicación + arquitectura cliente-servidor

Los 6 principios (constraints) de REST

1

Cliente-Servidor

Frontend y backend son independientes. El front sólo consume datos; el back sólo los sirve.

2

Sin estado (Stateless)

Cada petición es autónoma. El servidor NO recuerda peticiones anteriores. Toda la info va en el request.

3

Cacheable

Las respuestas pueden marcarse como cacheables. El cliente guarda la respuesta y no repite la petición.

4

Interfaz uniforme

URLs consistentes, métodos HTTP estándar, formato de respuesta previsible para todos los recursos.

5

Sistema en capas

Entre cliente y servidor puede haber proxies, load balancers, cachés. El cliente no lo sabe ni le importa.

6

Code on Demand

(Opcional) El servidor puede enviar código ejecutable al cliente (ej: JavaScript). Poco usado en la práctica.

Principio clave: Stateless (sin estado)

❌ Con estado (Stateful)

Request 1: "Hola, soy Juan"

Servidor: OK, recuerdo que sos Juan

Request 2: "Dame mis pedidos"

Servidor: Busco pedidos de Juan ✓

⚠️ **Problema:** si el servidor se reinicia
o hay otro servidor, perdiste el contexto.
No escala bien.

✅ Sin estado (Stateless)

Request 1:

GET /pedidos

Authorization: Bearer <token_de_juan>

Request 2:

GET /pedidos

Authorization: Bearer <token_de_juan>

✓ Cada request es auto-contenido.

🗣️ Analogía: como pedir en un restaurante donde el mozo nunca te conoció. Tenés que decir quién sos y qué querés cada vez.

Los verbos HTTP — qué acción realizan

GET

READ

Leer / obtener datos. No modifica nada.

GET /productos → Lista todos los productos

POST

CREATE

Crear un nuevo recurso en el servidor.

POST /productos → crea un producto nuevo

PUT

UPDATE

Reemplazar un recurso completo.

PUT /productos/5 → reemplaza el producto #5

PATCH

UPDATE

Modificar sólo algunos campos del recurso.

PATCH /productos/5 → actualiza sólo el precio

DELETE

DELETE

Eliminar un recurso del servidor.

DELETE /productos/5 → borra el producto #5

Recursos y URLs — la dirección de los datos

¿Qué es un recurso?

Un recurso es cualquier entidad del dominio que querés exponer:

- usuarios
- productos
- pedidos
- facturas

Cada recurso tiene su propia URL (dirección).
La URL identifica al recurso, NO a la acción.

🏠 Analogía: la URL es como la dirección de un edificio.
El método HTTP (GET/POST/etc.) es lo que hacés cuando llegás: entrar, salir, dejar algo.

Ejemplos de URLs bien diseñadas

GET	/api/productos	← Listar todos
GET	/api/productos/42	← Ver uno por ID
POST	/api/productos	← Crear uno nuevo
PUT	/api/productos/42	← Reemplazar #42
PATCH	/api/productos/42	← Editar parte de #42
DELETE	/api/productos/42	← Eliminar #42
❌ /api/obtenerProducto?id=42 /api/borrarProducto/42		

Códigos de estado HTTP — la respuesta del servidor

2xx Éxito

200 OK

La petición fue exitosa. Respuesta normal.

201 Created

Se creó un recurso nuevo (POST exitoso).

204 No Content

Éxito, pero sin contenido que devolver (DELETE).

4xx Error del cliente

400 Bad Request

La petición está mal formada o le faltan datos.

401 Unauthorized

No estás autenticado. Necesitás un token.

403 Forbidden

Autenticado, pero sin permisos para eso.

404 Not Found

El recurso solicitado no existe.

5xx Error del servidor

500 Internal Server Error

El servidor explotó. Bug en el backend.

503 Service Unavailable

El servidor está caído o saturado.

JSON — el idioma de las APIs REST

¿Qué es JSON?

JavaScript Object Notation.

Formato de texto ligero para representar datos estructurados.

- ✓ Legible por humanos
- ✓ Fácil de parsear
- ✓ Independiente del lenguaje
- ✓ Estándar de facto en APIs

No es código, es un FORMATO de intercambio de datos. Como el lenguaje que acuerdan usar el cliente y el servidor.



Analogía: es como el embalaje estándar para enviar paquetes. Siempre el mismo formato, cualquier lenguaje lo entiende.

Ejemplo de respuesta JSON

```
{
  "id": 42,
  "nombre": "Notebook Pro",
  "precio": 1299.99,
  "disponible": true,
  "categorias": [
    "electronica",
    "computacion"
  ],
  "vendedor": {
    "id": 7,
    "nombre": "TechStore"
  }
}
```

El ciclo completo: Request → Response



REQUEST (lo que enviás)

Método:

POST

URL:

/api/productos

Headers:

Content-Type: application/json
Authorization: Bearer eyJ...

Body:

```
{  
  "nombre": "Notebook",  
  "precio": 1299  
}
```



HTTP



RESPONSE (lo que recibís)

Status:

201 Created

Headers:

Content-Type: application/json
Location: /api/productos/43

Body:

```
{  
  "id": 43,  
  "nombre": "Notebook",  
  "precio": 1299,  
  "creadoEn": "2025-04-19"  
}
```

Cómo diseñar una API REST — reglas de oro

1



Usá sustantivos en plural, nunca verbos



GET /productos POST /usuarios DELETE /pedidos/5



GET /obtenerProductos POST /crearUsuario

2



Representá jerarquías con rutas anidadas



GET /usuarios/3/pedidos GET /pedidos/7/items



GET /getPedidosDelUsuario?id=3

3



El verbo HTTP hace el trabajo — no la URL



POST /productos (crear) DELETE /productos/5 (borrar)



POST /crearProducto POST /borrarProducto/5

4



Versioná tu API desde el inicio



GET /api/v1/productos GET /api/v2/productos



GET /productos (sin versión, cambios rompen clientes)



Una API bien diseñada es autodocumentada: cualquier desarrollador entiende qué hace sólo con leer la URL y el método.

¿Dónde van los datos? Path · Query · Body



Path Param

```
/recursos/{id}
```

Cuando identificás un recurso específico. Es parte de la identidad del recurso.

- Obligatorio siempre
- Identifica UNA entidad concreta
- Forma parte de la URL canónica

Ejemplos:

GET /usuarios/42

GET /pedidos/99/items/3

DELETE /productos/7



Query Param

```
/recursos?clave=valor
```

Filtros, búsquedas, paginación, ordenamiento. Opcionales por naturaleza.

- Siempre opcional
- No cambia el recurso base
- Múltiples separados por &

Ejemplos:

GET /productos?categoria=ropa

GET /usuarios?page=2&limit=20

GET /pedidos?estado=pendiente&orden=fecha



Request Body

```
{ "campo": valor }
```

Datos para crear o modificar un recurso. Sólo en POST, PUT y PATCH.

- Sólo POST / PUT / PATCH
- JSON en el body
- Nunca en GET o DELETE

Ejemplos:

```
POST /productos
{ "nombre": "Camisa" }
PUT /usuarios/5
{ "email": "x@y.com" }
PATCH /pedidos/3
{ "estado": "enviado" }
```



Errores comunes: Query para IDs · Body en GET · Path params opcionales · Mezclar filtros en el path

La regla para no equivocarse — cuándo usar cada uno



La pregunta clave:

"¿Esto IDENTIFICA el recurso, lo FILTRA, o lleva DATOS para modificarlo?"

Identificar → Path Param | Filtrar/Paginar/Ordenar → Query Param | Crear/Modificar datos → Body

	Path Param	Query Param	Body
¿Cuándo?	Identificar recurso concreto	Filtrar, paginar, ordenar	Crear o modificar datos
¿Obligatorio?	Siempre sí	Casi siempre no	Depende del endpoint
¿Métodos?	Todos	Principalmente GET	POST, PUT, PATCH
Ejemplo	/usuarios/42	/usuarios?rol=admin	{ "nombre": "Ana" }



Path = quién es



Query = cómo filtrarlo



Body = qué datos trae

Resumen — lo que tenés que recordar



Una API es un contrato: define qué podés pedir y qué vas a recibir. El mozo entre cliente y servidor.



REST usa HTTP como protocolo. Los verbos (GET/POST/PUT/DELETE) definen la acción sobre el recurso.



Cada recurso tiene una URL única. La URL nombra el QUÉ, el verbo HTTP el QUÉ HACER.



Stateless: el servidor no recuerda nada. Cada petición lleva todo lo necesario (token, parámetros, body).



JSON es el formato estándar de intercambio. Texto legible, estructurado, compatible con cualquier lenguaje.



Path=identificar, Query=filtrar, Body=datos. Nunca mezcles su propósito.

Una API REST bien diseñada es intuitiva, predecible y fácil de consumir por cualquier cliente 🚀